
UPAY · FOOTPATHSHALA

Operations Platform

Architecture, Technology Choices & Judge Q&A Prep

A working reference for explaining what was built, how it fits together, and why each technical decision was made — written so any answer a judge asks for is already here.

22
routes

109
unit tests

14
commits

6
AI-touched modules

The one-sentence pitch

UPAY's Footpathshala programme was losing about 1,600 staff-hours a month to manual data entry across disconnected spreadsheets. This is one platform — a Next.js app on Supabase Postgres, with Gemini and in-browser face recognition doing the two jobs a person previously did by hand — that records what happens at every centre and then answers questions about it in plain language.

Prepared as a companion document to the live deployment and the GitHub repository.

Contents

- | | |
|-----|--|
| 1. | The problem this solves |
| 2. | System architecture at a glance |
| 3. | Tech stack — and why not the alternative |
| 4. | Data model & access control |
| 5. | Deep dive — face-recognition attendance |
| 6. | Deep dive — UpayGPT (natural-language analytics) |
| 7. | Deep dive — AI assessment generation & OCR grading |
| 8. | Performance engineering: the RLS story |
| 9. | Security & privacy posture |
| 10. | Testing strategy |
| 11. | What's deliberately out of scope, and why |
| 12. | Judge Q&A; — rapid fire |

1. The problem this solves

UPAY runs Footpathshala — street-side learning centres for children who work or live near traffic signals and have little access to formal schooling. The programme spans multiple **zones**, each with several **centres**, each centre with its own **students** and **volunteers**. Attendance, volunteer hours, lesson delivery and assessment results were all tracked on spreadsheets that didn't talk to each other — a winning prior team's writeup put the resulting admin burden at roughly **1,600 staff-hours a month**, time that should have gone into teaching.

Three consequences of that fragmentation shaped every design decision in this build:

- **No single source of truth.** The same child's attendance might exist in a paper register at the centre and a different spreadsheet at head office, with no way to reconcile them.
- **No visibility into problems until they're severe.** A centre's attendance can decline for six weeks before anyone with the full picture notices.
- **Insight requires a person who can write a database query.** Field staff who could act on data had no way to ask it a question directly.

What “done” meant for this build

One system of record for the whole hierarchy (Zone → Centre → Student / Volunteer).
Attendance and volunteer verification captured with almost no manual typing.
Assessment generation and grading that don't require a teacher to author every question by hand.
Analytics that surface problems unprompted, plus a natural-language interface for anything else.

2. System architecture at a glance

The whole product is **one Next.js application** deployed to Vercel, talking to **one Supabase Postgres database**. There is no separate backend service — Next.js Server Actions and Route Handlers are the backend, running as serverless functions next to the Postgres connection pool. This collapses what would otherwise be three deployable services (API, database, auth) into one deploy target.

Layer	What runs there
Browser (volunteer / coordinator / admin)	React 19 client components — forms, the attendance capture flow, charts. Face detection (@vladmandic/face-api) runs here , entirely client-side.
Next.js server (Vercel)	App Router pages render server-side and call Server Actions for every mutation. Route handlers call Gemini for SQL generation, question papers, and OCR grading.
Supabase	Postgres 15 + Auth + Storage, reached through the `anon` key from the browser session and the `service_role` key only inside the seed script. Row-Level Security policies are the real authorization boundary — not the UI.
Google Gemini 2.5 Flash	Three jobs: turn a question into SQL, write an assessment paper, and read a photographed answer sheet. Called only from server code; the API key never reaches the browser.

Request path for a typical page (e.g. /dashboard)

- Browser requests /dashboard →
- Next.js middleware (proxy.ts) refreshes the Supabase session cookie, redirects to /login if absent →
- Server Component calls requireProfile() — one auth.getUser() + one profiles row, memoised per request with React's cache() — →

- Server Component queries Postgres views (`v_center_stats`, `v_attendance_weekly`, ...) through PostgREST, as the signed-in user → RLS silently narrows every row to what that role may see →
- React renders server-side; charts hydrate client-side with Recharts → HTML streamed to the browser.

Request path for an AI action (UpayGPT)

- User types a question in the browser →
- Server Action `ask()` sends the question + a hand-written schema description (no auth/storage tables mentioned) to Gemini →
- Gemini returns one `SELECT` statement →
- Application-side guard (`src/lib/sql-guard.ts`) rejects anything that isn't single-statement, read-only SQL →
- The vetted SQL is sent to Postgres via the `upaygpt_query()` SQL function, which re-checks the same rules server-side and runs under the caller's own RLS policies →
- Result rows go back to Gemini to be summarised in plain English and mapped to a chart type →
- Question, generated SQL, row count and answer are all written to an `ai_queries` audit table.

Why collapse everything into Next.js instead of a separate API service?

In an 18-hour build, a second service means a second deploy pipeline, a second auth story, and CORS — all pure plumbing that doesn't move the product forward.

Server Actions give type-safe, no-boilerplate calls from a form straight to a database mutation, with the framework handling the network layer.

The trade-off we accepted: this couples the web tier to Vercel's serverless runtime. For a programme this size that's the right trade — it would need revisiting only past a scale where a dedicated backend earns its complexity.

3. Tech stack — and why not the alternative

Every row below is a decision that had a real alternative on the table.

Chosen	Alternative considered	Why this one
Next.js 16 (App Router)	Create React App / plain Vite SPA	Server Components mean the dashboard's Postgres queries never ship a client-side data-fetching library; Server Actions remove the need to hand-write REST endpoints for every mutation.
Supabase (Postgres + Auth + Storage)	Firebase / hand-rolled Postgres + NextAuth	One signup gets a real relational database (needed for the Zone→Centre→Student joins and for UpayGPT's SQL), row-level security as a genuine authorization layer, and session auth — three problems solved by one service instead of three.
@vladmandic/face-api	Original face-api.js / a cloud vision API	The original face-api.js is unmaintained and pinned to TensorFlow.js 1.x, which breaks under modern bundlers. This fork runs on tfjs 4. A cloud API (AWS Rekognition, Azure Face) would mean uploading photographs of minors to a third party — unacceptable for this data.
Google Gemini 2.5 Flash	OpenAI GPT-4 / a rules-based generator	Gemini 2.5 Flash is fast and cheap enough for interactive use, and its multimodal input reads a photographed answer sheet directly — removing Tesseract OCR as a separate, weaker pipeline stage for handwriting specifically.
Tailwind CSS v4	CSS Modules / styled-components	Utility classes plus CSS custom properties gave one place (globals.css) to define the whole light/dark palette, referenced everywhere else by name instead of duplicated hex codes.
Recharts	D3 directly / Chart.js	A declarative React API kept every chart a thin wrapper around one shared, colour-blind-safe palette (see §6) rather than hand-tuned SVG per chart.
Zod	Manual if-checks / no validation	Every Server Action parses its input through a Zod schema before touching the database — the same defence whether the caller is the real UI or a crafted request.
Vitest	Jest	Near-zero config against a Vite-style toolchain and fast enough to run the full 109-test suite in under two seconds — kept the test loop tight during rapid iteration.

Full dependency list (as shipped)

next 16.3.2	react 19.2.8	react-dom 19.2.8
@supabase/supabase-js	@supabase/ssr	@google/genai
@vladmandic/face-api	recharts	leaflet / react-leaflet
jspdf	papaparse	zod
tailwindcss v4	class-variance-authority	lucide-react
vitest (dev)	tsx (dev)	postgres (dev, seed/migrate only)

4. Data model & access control

The schema mirrors the real organisation: **zones** → **centers** → **students**, with **profiles** (linked to Supabase Auth) carrying a role — admin, coordinator, teacher, volunteer, or student. Fourteen tables in total, plus four Postgres views that pre-aggregate the numbers every dashboard needs.

Core tables

zones ■■■ centers ■■■ students ■■■ attendance ■■■ class_sessions
profiles (role: admin | coordinator | teacher | volunteer | student)
volunteer_checkins · curriculum_units · center_curriculum
assessments · assessment_results · certificates · ai_queries

Row-Level Security: the access-control model actually lives in Postgres

The role matrix isn't just “hide the menu item.” Every table has RLS policies, so even a request that bypasses the UI entirely — a crafted API call, or a coordinator asking UpayGPT a cleverly-worded question — is still filtered at the database layer.

Role	Data scope	Can do
Admin	Every zone	Full read/write, issue certificates, import legacy data
Coordinator	Their own zone	Manage centres & volunteers in that zone, zone analytics
Teacher	Their own centre	Curriculum, assessments, attendance, student progress
Volunteer	Their own centre	Attendance capture, check-in, grade answer sheets
Student	Their own record	View their own progress only

Verified against the seeded dataset (queried directly, no application-level filter):

Account	Centres visible	Students visible
admin@upay.org	20 of 20	440 of 440
coordinator@upay.org	4 of 20 (own zone)	77 of 440 (own zone)
teacher / volunteer @upay.org	1 of 20 (own centre)	20 of 440 (own centre)

The UpayGPT-specific guardrail

A coordinator can ask UpayGPT literally “show me every student in the country” — the generated SQL still runs under their own Postgres session, so RLS still narrows the result to their 77 students. Verified directly: the query returned 77, not 440.

This matters because it means the AI layer cannot become a privilege-escalation path — the database enforces the boundary, not the prompt.

5. Deep dive — face-recognition attendance

The headline feature. A volunteer photographs the whole class once; the app marks the register. The distinctive design choice: **no photograph is ever uploaded, transmitted, or stored.** Recognition happens entirely on the volunteer's own device.

How it actually works

- 1. Detection.** The photo is decoded in the browser. SSD MobileNet v1 (via @vladmandic/face-api, model weights served from /public/models) finds every face and its bounding box.
- 2. Embedding.** A FaceNet-style recognition net converts each detected face into a **128-dimension vector** — a fixed-length numeric fingerprint of that face's geometry.
- 3. Matching.** For every student already enrolled at that centre, the app computes the Euclidean distance between the new face's 128-d vector and the student's stored vector. The closest match under a 0.5 distance threshold is treated as a hit; each student can only be claimed once (closest wins if two faces are ambiguous).
- 4. Human review.** The volunteer sees every match with a confidence percentage and can override any row — mark someone absent who matched, or present who didn't — before saving.
- 5. Enrolment loop.** A face that matches nobody is shown as a cropped thumbnail; naming it once both marks that child present *and* stores their embedding, so the very next class recognises them automatically.
- 6. What's saved:** only the 128 numbers, plus status/method/confidence. The photograph itself is discarded on the device the moment detection finishes — there is no column in the schema that could hold it.

Why this model, not a bigger one

SSD MobileNet v1 was chosen over a larger detector (e.g. a full ResNet-based face detector) because it has to run **on a volunteer's phone browser, on a weak connection, with no server cost per use.** It's the standard trade in face-api.js between accuracy and running entirely client-side; for a class of 15–25 children photographed head-on in daylight, it's more than accurate enough, and the alternative — a server-side model — would mean uploading every class photo, which the privacy design explicitly rules out.

The privacy answer, verbatim, for a judge who asks about storing children's faces

"The photograph never leaves the volunteer's device. Detection and the 128-number embedding are computed entirely in the browser; only that vector is sent to the server, and a vector cannot be reconstructed back into a face. The database schema has no image column for a student's face — it is structurally impossible to store one by accident."

6. Deep dive — UpayGPT (natural-language analytics)

A field coordinator shouldn't need to know SQL to ask “which centre is struggling?” UpayGPT is a text-to-SQL pipeline with two independent layers of defence around it, because a model that can write arbitrary SQL against a production database is a real attack surface if left unguarded.

The pipeline

```
question --> Gemini (given a hand-written schema, NOT introspected)
--> one SELECT/WITH statement
--> [Guard 1: application] src/lib/sql-guard.ts
--> [Guard 2: database] upaygpt_query() Postgres function
--> executes under the ASKING USER's own RLS session
--> rows --> Gemini again --> plain-English answer + chart spec
--> question, SQL, row count, answer all logged to ai_queries
```

Why the schema is hand-written, not introspected

Gemini is given a curated description of exactly fourteen tables and four views — it is **never told the auth or storage schemas exist**. A model can't generate a query against a table it doesn't know is there, which removes an entire class of prompt-injection risk before the SQL guard even runs.

The two-layer SQL guard

Rule	What it catches
Single statement only	Rejects a semicolon-separated second statement (e.g. ...; DROP TABLE ...)
SELECT / WITH only	Regex-anchored at the start of the (comment-stripped) query
Keyword denylist	insert, update, delete, drop, alter, create, truncate, grant, copy, execute, ...
Object denylist	pg_catalog, pg_sleep, information_schema, auth., storage., dblink, ...
Forced row limit + timeout	Database layer wraps the query in LIMIT 500, 5s statement_timeout, read-only transaction

Both layers exist on purpose: the application-side guard fails fast with a readable message; the database-side `upaygpt_query()` function is the layer that actually can't be bypassed, since it re-checks the same rules with no dependency on the client having behaved. It's covered by unit tests for the attacks that matter — stacked statements, keywords hidden inside comments, reads of the auth schema, `pg_sleep` denial-of-service, `dblink` exfiltration.

Why show the generated SQL to the user at all?

An answer nobody can audit is not a trustworthy answer. The “see the SQL I ran” panel is the feature, not a debug leftover — it's what turns a black-box chatbot into something a coordinator can actually verify and trust before acting on it.

7. Deep dive — AI assessment generation & OCR grading

Generation

Gemini writes a paper for a **learning level** (foundation → bridge), not a school grade — these children's schooling is interrupted, so grade doesn't map to ability. The system prompt constrains content to what a footpath-schooled child actually recognises: Indian names, rupees, autos, mangoes — explicitly never a home computer, a garden, or a foreign context. About two-thirds of questions are generated as multiple-choice on purpose, because MCQ is dramatically more reliable to read back from a photographed answer sheet than free text.

Grading — deliberately split into two separate concerns

This is the single most important design decision in the assessments module, and worth stating precisely if a judge asks how grading avoids AI hallucination:

Stage	Who does it	Why
Reading	Gemini's vision model (multimodal, given the photographed sheet)	Reports <i>exactly</i> what is written. Explicitly instructed not to correct spelling or guess unclear handwriting — an unreadable answer is flagged illegible, never silently marked wrong.
Grading	Plain TypeScript (src/lib/grading.ts), zero AI involved	Ordinary string/number comparison: normalises "(C)", "3" and "C." to the same option letter; strips rupee signs from numeric answers; matches short answers against an accepted-alternatives list (e.g. "kela" for "banana").

Because grading is ordinary code, not a model call, it is fully unit-testable — and is: option-letter normalisation, rupee-sign stripping, accepted-alternative matching and blank-answer handling all have dedicated test cases.

Nothing reaches a child's permanent record until a volunteer reviews and confirms it on a verification screen; the stored total is recomputed from the volunteer's confirmed rows, never trusted from the model's own arithmetic.

The line to use if asked “isn't AI grading risky?”

“The AI never grades anything. It transcribes handwriting, and transcription is the one part of this pipeline vision models are genuinely reliable at. Deciding right-or-wrong is ordinary, testable code — and even that result is a suggestion until a human confirms it.”

8. Performance engineering: the RLS story

Worth including because it's the strongest evidence of real engineering rigour in this build — a bug that was found, root-caused with EXPLAIN ANALYZE, and fixed with a measured before/after, not guessed at.

Early load testing showed the dashboard taking 2.8–8.7 seconds, and two analytics queries **timing out completely** against Postgres's 8-second statement limit. The raw queries executed in 12–234ms as the table owner (RLS bypassed) — so the query plans were fine. The problem was specific to how RLS was evaluating them for a real, authenticated user.

Root cause, from EXPLAIN ANALYZE

```
Seq Scan on attendance (actual time=589.582..601.739 rows=40858)
  Filter: (ANY (session_id = (hashed SubPlan 2).col1))
  SubPlan 2
    -> Seq Scan on class_sessions cs (actual time=2.501..587.115 rows=1857)
        Filter: (can_touch_center(center_id) AND can_touch_center(center_id))
Execution Time: 604.987 ms          <- 587ms of that was the filter alone
```

`can_touch_center(center_id)` was a function call inside the RLS policy — and Postgres **cannot hoist a function call out of a row-by-row filter**, so it ran once per scanned row (twice, in fact, because the joined table's own policy applied as well). On roughly 40,000 attendance rows, that's tens of thousands of function calls to answer a query that returns twenty aggregate rows.

The fix: rewrite the predicate as a set, not a per-row test

Changed every scoped RLS policy from a function-call predicate to a set-membership predicate:

	Predicate	Cost
Before	using (<code>can_touch_center(center_id)</code>)	Evaluated per row — cannot be hoisted
After	using (<code>center_id in (select visible_center_ids())</code>)	Evaluated once into a hashed InitPlan; each row then costs one hash lookup

Query	Before	After	Change
<code>v_center_stats</code>	5,292 ms	462 ms	91% faster
<code>v_student_progress</code>	timed out (>8s)	382 ms	fixed
<code>v_attendance_daily</code>	timed out (>8s)	393 ms	fixed
attendance row count	1,509 ms	105 ms	93% faster

Two secondary fixes landed alongside this: PostgREST's default 1,000-row cap had been silently truncating the 120-day attendance trend, so a weekly rollout `v_attendance_weekly` view now aggregates in Postgres instead of the client fetching every row; and `requireProfile()` was running twice per request (once in the layout, once in the page) until wrapped in `React's cache()`.

Net result, measured against the production build

Every page now renders in 340–760 ms, down from 2.8–8.7 seconds — and two pages that were failing outright now succeed. A custom smoke-test script (`scripts/smoke.ts`) renders all 16 authenticated routes and asserts on actual page content, not just an HTTP 200, after this bug also revealed two pages silently rendering an empty state due to an ambiguous foreign-key relationship.

9. Security & privacy posture

Concern	Approach
Authorization	Row-Level Security in Postgres on every table, not just UI-level route guards. Verified programmatically per role (§4).
Face data	128-d embeddings only; no photograph is ever transmitted or stored (§5).
Secrets	service_role key (bypasses all RLS) used only in the local seed script, never in any code path the browser can reach; .env.local is git-ignored.
SQL injection / AI misuse	Two-layer guard on all AI-generated SQL, tested against stacked statements, comment-hidden keywords, auth-schema reads, pg_sleep, dblink (§6).
Input validation	Every Server Action parses its input through a Zod schema before it reaches the database.
Geo-verification	Volunteer check-in distance is recomputed server-side from the reported coordinates — a device can claim any location, but not how far that location is from the centre.
Session handling	Supabase Auth (JWT-based) with middleware-refreshed cookies; every non-public route redirects unauthenticated requests to /login.

10. Testing strategy

109 Vitest unit tests, deliberately aimed at logic that fails *quietly* rather than loudly — the class of bug that a working demo won't surface on its own:

Area	Cases	What it defends against
SQL guard	18+ cases	Injection attempts, comment-hidden keywords, forbidden objects
Face matching	13	Descriptor distance, threshold behaviour, confidence scaling
Grading	20+	Option-letter parsing, numeric tolerance, accepted-alternative answers
Certificates	15	Eligibility thresholds, tenure requirements, edge cases (0 shifts, negative hours)
CSV import mapping	20+	Header-guessing against real messy spreadsheet headers, date parsing (dd/mm/yyyy and ISO), phone normalisation
Geofencing / utils	13	Haversine distance, percentage helpers

Beyond unit tests, `scripts/smoke.ts` signs in as a real seeded user and renders every authenticated route, asserting on rendered page text — written specifically after discovering that a status-code-only check had missed two pages rendering a valid, empty 200 response.

11. What's deliberately out of scope, and why

Anticipate this question — judges often probe scope decisions directly:

Left out	Why
Native mobile app	The deliverable is a responsive website; the interface works on a phone browser today. A native app is a natural next phase, not a missing requirement.
Offline-first sync	Attendance queues in localStorage and retries on reconnect if a submit fails, but there's no full offline-first architecture (service worker, conflict resolution) — a deliberate scope cut for an 18-hour build.
SMS/WhatsApp parent notifications	Designed for but not built; the data model (guardian_phone on every student) already supports adding it without a schema change.
Payment / donation handling	Out of scope entirely — this is an operations platform, not a fundraising one.

12. Judge Q&A; — rapid fire

Answers phrased the way you'd actually say them on stage.

“What happens if the model hallucinates a wrong SQL query in UpayGPT?”

It's caught before it runs. The generated SQL passes an application-side guard, then a database-side guard that re-checks independently, and even a syntactically valid but semantically wrong query just returns wrong-looking data with its SQL visible in the ‘see the SQL I ran’ panel — so the wrongness is auditable, not hidden.

“Couldn't a volunteer just fake a check-in from home?”

The browser reports GPS coordinates, but the server — not the device — computes the distance to the centre using those coordinates. A shift outside the centre's radius is logged as *unverified* rather than silently accepted or rejected, so the record stays honest either way.

“Why Supabase and not a custom backend?”

Because Postgres RLS gave us a real, enforced authorization boundary — not just hidden UI — with zero backend code, and one signup covered database, auth, and file storage together. The performance work in §8 shows we understand exactly what that trade costs and how to fix it when it bites.

“Is the face recognition actually accurate?”

It uses the same detector/embedding pair (SSD MobileNet + a FaceNet-style recognition net) that face-api.js's own accuracy benchmarks are built on, matched at a Euclidean-distance threshold of 0.5. It's not perfect — which is exactly why every match ships with a confidence score and a human can override any row before saving.

“What's the single riskiest part of this system?”

Honestly, RLS policy correctness — get one wrong and data leaks across zones. That's why §4 isn't just a claim; it's a table of numbers verified by querying the seeded database directly, per role, with no application code in the loop.

“How would this scale to all of UPAY's programmes, not just Footpathshala?”

The schema is already generic — zone/centre/student/volunteer isn't Footpathshala-specific vocabulary. The main scaling question is Postgres row counts, and §8 shows the query layer was already re-engineered once under real load to stay fast.

“What did you personally build vs. what did AI generate?”

Every architectural decision, every RLS policy, the performance debugging, and the review of every AI-touched code path were done by the team; Gemini is a runtime dependency of the product itself (not a code-generation shortcut) — it writes SQL, question papers, and OCR transcriptions *at request time*, the same way it will in production.

UPAY Footpathshala Operations Platform — built to give the hours back to the children.